SOFTWARE REQUIREMENTS SPECIFICATION — VERSION 2.0

SANJHI

سانجھی

Trust, digitized. Your committee, without the WhatsApp chaos.

Document Title: Sanjhi — Software Requirements Specification & Gap Analysis

Version: 2.0 (Consolidated with Production Architecture Audit)

Supersedes: Version 1.0, August 2026

Prepared For: Sanjhi Product & Engineering Team

Platform Scope: Web Application + Native Android APK (Capacitor) + iOS (React Native) + WhatsApp Conversational Interface

Includes full production architecture & SRS gap audit

Table of Contents

What's New in Version 2.0

This revision merges the original v1.0 SRS (Sections 1–8, unchanged as the contractual baseline) with a new Production Gap Analysis and Architecture Audit (Sections 9–16) documenting how the shipped implementation compares to the original spec, including nine new architecture diagrams: a C4 context diagram, a layered software architecture, a full ERD, an AI dispute-resolution sequence diagram, a WhatsApp voice pipeline flowchart, two financial/trust state machines, an offline-cache flowchart, and a consolidated roadmap mind map.

Document Control —

Part I — Original Software Requirements Specification (v1.0)

1. Introduction 3

2. Overall Description —

3. Functional Requirements (3.1–3.14) —

4. Non-Functional Requirements —

5. External Interface Requirements —

6. Design Section — Screens & Wireflows —

7. Future Enhancements (Phase 2 / Phase 3) —

8. Original Open Assumptions Log —

Part II — Production Gap Analysis & System Architecture (New in v2.0)

9. Gap Analysis & Production Implementation Overview —

10. System Architecture — High-Level Context (C4 + Layered) —

11. Database Design — Entity-Relationship Diagram —

12. AI Agent Architecture (Dispute Court + WhatsApp Voice) —

13. Financial & Trust State Models —

14. Client Performance Architecture (Offline Cache) —

15. Future Roadmap — Consolidated View —

16. Summary of Key Takeaways —

Document Control

Field	Detail
Document Title	Sanjhi — Software Requirements Specification
Version	2.0 — Consolidated SRS + Production Gap Analysis (supersedes v1.0, August 2026)
Prepared For	Sanjhi Product & Engineering Team
Platform Scope	Web Application + Mobile Application (Android native APK via Capacitor & iOS via React Native) + WhatsApp Conversational Interface (text & voice)
Related Document	Sanjhi Product Proposal (Executive Summary)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for Sanjhi, a web and mobile platform that digitizes the traditional committee (bisi / kameti / ROSCA) savings system practiced widely across Pakistan and South Asia. This document is intended for the product, design, and engineering teams building the platform, and serves as the single source of truth for what the system must do in its first release (MVP) and its planned Phase 2 evolution.

1.2 Scope

Sanjhi allows any registered user to create or join a rotating savings committee, track member contributions transparently, and manage payouts on a fixed schedule. The platform does not hold or move money itself — funds move directly between members and the organizer's linked account, while Sanjhi provides tracking, reminders, a privacy-respecting transparency ledger, and reconciliation on top. Payments in this release are Manual only: the Organizer's "Mark as Paid" action, informed by the member-submitted sender details, is the fallback of record.

In addition to the web and mobile apps, Sanjhi offers an optional WhatsApp-based conversational interface (Section 3.14) that lets an already-registered or newly onboarding user log in or sign up, and then check committees, make payments, join a committee, or file a complaint, entirely through natural-language chat — without opening the app.

The system supports two primary in-app roles (Organizer and Member), an elevated Co-Organizer permission level, and a platform-level Admin role. This SRS covers authentication, committee lifecycle, invitations, payment tracking, notifications, an AI-assisted FAQ chatbot, natural-language committee setup, two purpose-built AI agents (an investigative Complaint Case-Builder Agent and a proactive, background Committee Health Monitor Agent), and a WhatsApp Conversational Agent that exposes the same core actions through a chat-based, tool-calling assistant.

Database schema design is intentionally out of scope for this document.

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
Committee	A rotating savings group (also called bisi, kameti, or ROSCA) where members contribute a fixed amount on a schedule and one member receives the pooled payout each cycle.
Organizer	The user who creates a committee and is the default owner/administrator of that committee.
Co-Organizer	A member promoted by the Organizer to share operational control of a committee, capped below full ownership rights.
Member	A participant in a committee who contributes funds and receives a payout on their scheduled turn.
Cycle	One collection period within a committee's schedule (e.g., one month), at the end of which contributions are reconciled and a payout is released.
Interval	The frequency at which members must pay into the committee (e.g., every 15 days, monthly, every 2 months).
Duration	The total lifespan of a committee, calculated as Capacity × Interval for fixed-order payout committees.

Term	Definition
Ledger	The real-time record of committee payment progress. Organizer/Co-Organizer see every member's status; a Member sees only their own history plus an anonymized committee-wide progress count.
Trust Score	A portable reputation metric reflecting a user's payment reliability across all committees they have joined.
Risk Flag	A private, per-cycle prediction shown only to Organizer/Co-Organizer indicating a member may be about to miss a payment (see FR-TRUST-02).
OTP	One-Time Password, used to verify a phone number or email during signup and login, including WhatsApp-based login/signup (Section 3.14).
Agent	A Gemini-orchestrated (or, for the WhatsApp Conversational Agent, Groq-orchestrated) workflow that chains multiple tool calls and reasoning steps to produce a structured output — distinct from a single one-shot prompt/response call.
Tool Call	A structured request an agent makes to Sanjhi's own backend (e.g., "get ledger history for user X in committee Y") to fetch data or perform an action it needs mid-task.
Case File	The structured investigative output of the Complaint Case-Builder Agent: narrative summary, full evidence trail, contradiction findings, and a recommended action (see FR-SUPPORT-02).
Health Memory	The per-committee state the Committee Health Monitor Agent persists across runs (historical baselines, prior recommendations) so it can reason about trends, not single-point readings (see FR-MONITOR-01).
MCP (Model Context Protocol)	A structured tool layer that exposes a defined set of Sanjhi backend capabilities (e.g., get_my_committees, make_payment) as callable "tools" for an AI agent, so the agent can act on a user's behalf without the agent ever holding direct database access (see Section 3.14).
Conversational Session	The per-user authentication and task state that the WhatsApp Conversational Agent maintains across a sequence of chat messages, tracking whether the user is unauthenticated, mid-OTP verification, or fully authenticated (see FR-WABOT-01).

1.4 Technology Stack

Layer	Technology
Frontend (Web)	React
Frontend (Mobile)	React Native
Backend	Node.js with Express
Database	PostgreSQL
AI / Gemini usage	Google Gemini API (free tier) — FAQ chatbot, natural-language committee setup parsing, and the Complaint Case-Builder Agent's tool-calling investigation and narrative synthesis (Section 3.11)
Agent orchestration	Backend-hosted agent logic coordinating Gemini calls with internal tool calls (Ledger, Trust Score/Risk Flag reads) for the Case-Builder Agent; a disclosed rules-based signal engine (no Gemini call) for the Health Monitor Agent (Section 3.13)
WhatsApp Conversational Interface	WhatsApp connectivity (Baileys) for receiving and sending chat messages; Groq API (LLM function-calling) as the intent-parsing and tool-selection agent; a Sanjhi MCP tool layer that wraps existing REST endpoints so the agent can only invoke actions the authenticated user is already permitted to perform (Section 3.14)

Layer	Technology
Payments (v1)	Manual tracking against organizer-linked JazzCash / Easypaisa / bank account. No aggregator integration in this phase.
Notifications	SMS gateway, WhatsApp Business API, and mobile push notifications
Infrastructure	Cloud-hosted (AWS / DigitalOcean); scheduled jobs (cron / queue-based) for reminders, cycle rollovers, risk-flag evaluation, and the Committee Health Monitor Agent's periodic scans

1.5 Document Conventions

- Each functional requirement is tagged with an ID (e.g., FR-AUTH-01) for traceability.
- "Shall" indicates a mandatory MVP requirement. "Should" indicates a strong recommendation. "Phase 2" marks a planned but deferred capability.
- Wireflows describe the screen-to-screen (or, for the WhatsApp Conversational Agent, message-to-message) navigation path for a use case; the Design section (Section 6) details individual screen contents.

2. Overall Description

2.1 Product Perspective

Sanjhi is a new, standalone product that replaces the informal, WhatsApp-and-memory-based coordination of committee groups with a structured digital workflow. It does not replace JazzCash, Easypaisa, or bank transfer as the actual movers of money — it sits alongside them as a coordination, transparency, and trust layer. This positioning keeps the product free of the regulatory and licensing overhead that comes with holding customer funds, while still solving the core pain points: manual tracking, awkward reminders, and lack of a shared, verifiable, yet privacy-respecting record.

Notably, Sanjhi also meets users where the informal version of this workflow already lives: WhatsApp. The WhatsApp Conversational Agent (Section 3.14) lets a user log in or sign up and perform core tasks directly inside a WhatsApp chat, using the same backend, the same permissions, and the same trust rules as the web and mobile apps — it is a new interface onto Sanjhi, not a parallel product.

2.2 User Classes & Characteristics

Role	Description	Scope of Control
Admin	Platform-level staff account. Not tied to any single committee.	Manage all users; view all committees; triage and handle complaints using agent-built Case Files (FR-SUPPORT-02); suspend accounts; freeze committees.
Organizer	The user who created a committee. Default owner.	Full control of that committee: settings, approvals, full ledger, marking payments received, promoting co-organizers, closing the committee, and acting on Health Monitor recommendations (FR-MONITOR-01).
Co-Organizer	A member promoted by the Organizer to assist with running a committee.	Same operational rights as Organizer (approvals, full ledger, reminders, marking payments) except cannot remove/replace the founding Organizer, or delete the committee.
Member (Participant)	A user who has joined a committee to contribute and receive a payout.	View own payment history and an anonymized committee progress count; make payments; view own trust score; request to join other committees.

A single user account can hold different roles across different committees simultaneously. Role is stored per committee-membership, not globally on the user profile. This role model is enforced identically regardless of which interface — web, mobile, or the WhatsApp Conversational Agent — the user is acting through (see FR-WABOT-02).

2.3 Operating Environment

- Mobile app: Android and iOS, via React Native.
- Web app: modern evergreen browsers (Chrome, Safari, Edge, Firefox), via React.
- WhatsApp Conversational Interface: any WhatsApp client (mobile or WhatsApp Web) capable of standard text messaging — no separate install beyond WhatsApp itself is required (Section 3.14).
- Backend services hosted on cloud infrastructure with horizontal scalability for scheduled jobs (reminders, cycle rollovers, risk-flag evaluation, and Committee Health Monitor scans).

2.4 Assumptions & Dependencies

- Users have a valid phone number capable of receiving SMS/OTP; email is offered as an alternate signup method.
- Organizers are willing and able to link a personal JazzCash, Easypaisa, or bank account as the committee's collection point.
- Google Gemini API free-tier rate limits are sufficient for FAQ chatbot, complaint case-building, and NL-setup parsing combined; usage is monitored and each feature scoped narrowly to stay within limits. The Committee Health Monitor Agent is designed to run on rules-based signals precisely so it adds no extra Gemini load (Section 3.13).
- Users of the WhatsApp Conversational Agent already possess (or are willing to create) a WhatsApp account on the number they use for Sanjhi; the bot's own OTP delivery depends on that WhatsApp session remaining reachable (Section 3.14).

2.5 Constraints

- Sanjhi does not hold or escrow funds; all money movement happens directly between members and the organizer's own linked account.
- Payments are Manual only in this release — there is no automatic payment detection, no in-app checkout, and no aggregator integration. The Organizer's manual "Mark as Paid" action is the fallback of record.
- Identity verification is limited to phone number and OTP; no CNIC or document-based KYC is collected, including for accounts created or logged into via the WhatsApp Conversational Agent.
- Individual member payment status is confidential between that member and the Organizer/Co-Organizer running the committee; it is not broadcast to the rest of the group (see FR-LEDGER-01), and this confidentiality rule applies identically to responses the WhatsApp Conversational Agent gives a Member.
- Both AI agents from the original release are advisory only: the Case-Builder Agent never resolves a complaint on its own, and the Health Monitor Agent never takes action on a committee — a human always makes the final call (see Sections 3.11 and 3.13). The WhatsApp Conversational Agent is different in kind — it executes real, user-authorized actions (e.g., submitting a payment confirmation) rather than only advising — so every action it takes is scoped to that user's own existing permissions and is never broadcast or auto-approved beyond what the same user could already do in the app (see FR-WABOT-02).

3. Functional Requirements

This section details every feature of Sanjhi, organized by functional area. Each requirement lists the actors involved, preconditions, the main flow, alternate/exception flows, and governing business rules.

3.1 Authentication & Account Management

FR-AUTH-01 Sign Up with Phone or Email

A new user shall be able to create a Sanjhi account using either a phone number or an email address, with phone number as the primary and recommended method given local usage patterns.

Actors: New User *Preconditions:* User does not already hold a Sanjhi account with the given phone number or email.

Main Flow:

1. User opens the app and selects "Sign Up." 2. User chooses to continue with Phone Number or Email. 3. User enters their phone number (or email) and taps Continue. 4. System sends a 6-digit OTP via SMS (or email). 5. User enters the OTP within the validity window (5 minutes). 6. On successful verification, user sets a display name and optional profile photo. 7. Account is created and user is taken to the Dashboard.

Alternate / Exception Flows:

- Invalid/expired OTP → system shows an error and offers "Resend OTP" (rate-limited to prevent abuse). - Phone/email already registered → system redirects to Login instead of Sign Up.

Business Rules:

- OTP is mandatory for both phone and email signup paths. - No CNIC or government ID is collected — phone/email + OTP is the only identity verification.

FR-AUTH-02 Log In & Persistent Session

A returning user shall be able to log in using their registered phone/email plus OTP, and remain logged in across app restarts until they explicitly log out.

Actors: Returning User *Preconditions:* User has a previously created account.

Main Flow:

1. User enters phone number or email. 2. System sends OTP; user enters it. 3. System verifies OTP and issues a session (refresh token pattern). 4. User lands on Dashboard; session persists on the device until logout or token revocation.

Alternate / Exception Flows:

- Wrong OTP entered 3+ times → temporary cooldown before another OTP can be requested.

Business Rules:

- Sessions persist by default; no separate "remember me" toggle is required since OTP-based login has no password to 'remember' in place of.

3.2 Committee Creation

FR-CC-01 Create a Committee

Any logged-in user shall be able to create a new committee by defining its financial terms, schedule, and collection account. The creator becomes the default Organizer.

Actors: Organizer (creator) *Preconditions:* User is logged in and has completed profile setup.

Main Flow:

1. User taps "Create Committee" from the Dashboard. 2. User enters committee name, contribution amount per member, and capacity (total number of members) — or uses "Describe it instead" (FR-CC-02) to pre-fill these fields. 3. User selects the collection Interval from a dropdown: every 15 days, every 1 month, or every 2 months. 4. System auto-calculates and displays total Duration = Capacity × Interval. 5. User selects payout order (Fixed order in MVP; Lottery and Bidding are Phase 2). 6. User links the committee's collection account (JazzCash, Easypaisa, or bank account) that all members will pay into. 7. User reviews a summary screen and confirms. 8. System creates the committee, generates an invite code and invite link, and opens the Committee Detail screen.

Alternate / Exception Flows:

- User abandons setup before confirming → draft is discarded; no partial committee is created.

Business Rules:

- Duration is auto-calculated (Capacity × Interval) and is not independently editable for fixed-order committees. - Only one collection account may be linked per committee at a time; it can be changed later by the Organizer, which updates the account shown to all members going forward.

FR-CC-02 Natural-Language Committee Setup

The system shall let an Organizer describe a committee in plain language (typed or dictated) and have the Committee Setup Form pre-filled automatically for review, instead of manually filling each field one by one.

Actors: Organizer (creator) *Preconditions:* User has selected "Create Committee."

Main Flow:

1. On the Committee Setup Form, user sees an optional "Describe it instead" field/toggle, with placeholder text like "e.g., 10 people, Rs. 5,000 monthly, starting next month." 2. User types (or dictates via voice-to-text) a free-text description and taps "Parse." 3. System sends the text to the Gemini API with a prompt constrained to Sanjhi's committee schema (name, amount, capacity, interval, start date) and receives structured field values back. 4. Parsed values pre-fill the normal Setup Form fields — name, amount, capacity, interval — exactly as if the user had typed them manually. 5. User reviews and edits any field, then continues through Schedule, Link Account, and Review & Confirm (FR-CC-01) unchanged.

Alternate / Exception Flows:

- Gemini cannot confidently parse a field (e.g., an ambiguous amount) → that field is left blank and highlighted for manual entry rather than guessed. - Gemini API is unavailable or rate-limited → the "Describe it instead" option is hidden/disabled; the standard manual form still works with no loss of functionality.

Business Rules:

- Parsed output is always a draft pre-fill, never auto-submitted — the mandatory Review & Confirm step in FR-CC-01 still applies. - This is a convenience layer only; it does not change the underlying committee schema, validation, or business rules (Duration = Capacity × Interval, etc.). - Why it matters: this removes the most tedious part of onboarding — typing the same details into five separate fields — and is a fast, demoable Gemini integration on top of infrastructure the product already has.

3.3 Invitations & Joining

FR-JOIN-01 Invite Members

The Organizer shall be able to invite people to a committee via a shareable invite code/link, or by directly adding a known Sanjhi user by their User ID.

Actors: Organizer / Co-Organizer *Preconditions:* Committee exists and has open capacity.

Main Flow:

1. From the Committee Detail screen, Organizer taps "Invite." 2. System displays the current invite code and a shareable link, always visible in the committee's description/detail area. 3. Organizer shares the code/link via any channel (WhatsApp, SMS, etc.), or alternatively searches for a user by User ID and adds them directly.

Alternate / Exception Flows:

- Invite link has expired → Organizer taps "Regenerate Invite" to issue a new code/link at any time. - Committee is at full capacity → invite option is disabled and shows "Committee Full."

Business Rules:

- Invite links/codes expire 24 hours after generation and can be regenerated on demand by the Organizer or Co-Organizer. - Directly adding a user by User ID skips the join-approval step described in FR-JOIN-02, since the Organizer is the one initiating the addition.

FR-JOIN-02 Join a Committee

A logged-in user shall be able to request to join a committee either by entering an invite code or opening an invite link, subject to Organizer approval.

Actors: Prospective Member *Preconditions:* User is logged in; committee has open capacity; invite code/link is valid (not expired).

Main Flow:

1. User taps "Join Committee" from the Dashboard. 2. User enters the invite code, or opens the invite link directly. 3. System shows a read-only Committee Preview (name, amount, interval, duration, Organizer name). 4. User taps "Request to Join." 5. Request enters a Pending state and is sent to the Organizer for approval. 6. Once approved, the committee appears on the Member's Dashboard and the Member gains access to the Committee Detail screen.

Alternate / Exception Flows:

- Invite code/link expired → user is shown an error and prompted to request a fresh invite from the Organizer. - Organizer rejects the request → user is notified and the slot remains open.

Business Rules:

- A join request does not occupy a capacity slot until the Organizer approves it — this prevents a leaked code from silently filling the committee.

3.4 Roles & Permissions

FR-ROLE-01 Promote a Co-Organizer

The Organizer shall be able to promote any existing Member of a committee to Co-Organizer, granting them operational control equivalent to the Organizer, with one ownership-level exception.

Actors: Organizer *Preconditions:* Target user is already an approved Member of the committee.

Main Flow:

1. Organizer opens the Member list on the Committee Detail screen. 2. Organizer selects a Member and taps "Make Co-Organizer." 3. System updates that Member's role for this committee only — their role in any other committee is unaffected.

Alternate / Exception Flows:

- Organizer can also demote a Co-Organizer back to Member at any time.

Business Rules:

- Co-Organizer inherits full operational rights: approve/reject join requests, view and edit the full ledger, mark payments received, send reminders, edit committee settings. - Co-Organizer cannot remove or replace the founding Organizer, and cannot delete/close the committee — these actions remain exclusive to the founding Organizer. - Role is scoped per committee-membership record, allowing a single user to be Organizer of one committee and Member (or Co-Organizer) of another simultaneously.

3.5 Payment Collection & Tracking

This is the core trust-building feature of Sanjhi. Because the platform does not hold funds (Section 2.5), payment tracking is Manual in this release: the Member pays the Organizer's linked account directly outside the app, submits the sender details for matching, and the Organizer/Co-Organizer confirms receipt. There is no Automatic/aggregator mode in this version — see Section 7 for that as an undated future item.

FR-PAY-01 Manual Payment & Reconciliation

The Member sends funds directly to the Organizer's linked account outside the app, and the system reconciles the ledger using submitted sender details, falling back to the Organizer's manual confirmation.

Actors: Member, Organizer/Co-Organizer *Preconditions:* Collection cycle is due.

Main Flow:

1. System notifies the Member that a payment is due and displays the Organizer's linked collection account details (account/wallet number, account title). 2. Member sends the contribution using their own JazzCash, Easypaisa, or bank app. 3. Member submits the sending account details in-app (which account/number the transfer was sent from), so the transaction can be matched. 4. System attempts to auto-confirm receipt where technically possible; if it cannot, the payment remains "Awaiting Confirmation." 5. Organizer or Co-Organizer reviews the (full) ledger and manually marks the Member as Paid once they've verified receipt in their own account — this is the fallback path used for most transactions. 6. The Member's own payment record updates immediately in their private "My Payments" view (FR-LEDGER-01); the committee-wide anonymized progress count updates for all members.

Alternate / Exception Flows:

- Member disputes a "not paid" status they believe is incorrect → Member can file a complaint (Section 3.11) with evidence (e.g., transaction screenshot) for Admin review.

Business Rules:

- Sanjhi never touches or holds the funds — the transaction happens entirely between Member and Organizer's own account. - Manual mark-as-paid by the Organizer/Co-Organizer is treated as the authoritative confirmation of record. - There is no payment-mode toggle in this release — Manual is the only mode, so every Member follows this same flow.

3.6 Payout Distribution

FR-PAYOUT-01 Release the Cycle Payout

Once all Members are marked Paid for a given cycle, the system shall identify that cycle's scheduled recipient and notify the Organizer to release the pooled amount.

Actors: Organizer, Member (recipient) *Preconditions:* All Members marked Paid for the current cycle.

Main Flow:

1. System detects that 100% of Members are marked Paid for the cycle. 2. System identifies the recipient based on the committee's payout order (Fixed order in MVP). 3. Organizer is notified to send the pooled amount to that cycle's recipient (money moves directly, as with collection). 4. Organizer marks the payout as Sent once complete; recipient can confirm receipt. 5. System advances the committee to the next cycle and resets the ledger for the new collection period.

Alternate / Exception Flows:

- Not all Members paid by the due date → payout is held pending; see FR-PAYOUT-02.

Business Rules:

- A Member's payout turn is not affected by their own payment status for that same cycle — they still owe their own contribution regardless of receiving the payout that cycle.

FR-PAYOUT-02 Handling Missed Payments (Default)

When a Member misses their payment deadline, the system shall flag the default clearly to the people who need to act on it, and leave resolution to the Organizer and Members, rather than auto-resolving it.

Actors: Organizer, Member *Preconditions:* Cycle due date has passed with one or more Members unpaid.

Main Flow:

1. System flags the unpaid Member(s) as "Overdue," by name, on the full ledger — visible to the Organizer/Co-Organizer only. 2. All other Members instead see a neutral, non-identifying banner (e.g., "This cycle is still collecting — payout may be delayed") without names attached. 3. System notifies the Organizer and the specific overdue Member directly. 4. Organizer and the overdue Member resolve the situation directly (e.g., extend the deadline, delay the payout, or escalate via a complaint) — the system does not automatically delay, cancel, or reallocate the payout.

Business Rules:

- This mirrors how real-world committees already handle defaults — socially, between the Organizer and the individual — and avoids the platform either making financial decisions it has no authority to enforce, or exposing one member's payment shortfall to the whole group.

3.7 Ledger & Transparency

Payment status is personal financial information, so the ledger's visibility is scoped by role: Organizer/Co-Organizer see everything they need to run the committee and confirm payments; Members see only their own history plus a non-identifying view of the group's progress.

FR-LEDGER-01 Ledger Visibility (Role-Scoped)

The system shall show two different views of committee payment status depending on role: Organizer/Co-Organizer see the full, per-member ledger for management purposes; Members see only their own payment history plus a non-identifying, aggregate view of the committee's progress.

Actors: Organizer, Co-Organizer, Member *Preconditions:* User is an approved member of the committee.

Main Flow:

1. Organizer/Co-Organizer opens the Ledger tab and sees every member's name, payment status (Paid / Awaiting Confirmation / Overdue) for the current cycle, and history of past cycles, since they need this to run the committee and confirm payments. 2. A Member instead opens "My Payments" and sees only: their own status and history for every cycle they've been part of (in this and other committees), their own next due date and amount, and their own upcoming

payout turn. 3. In the Committee Detail screen, a Member additionally sees a simple aggregate progress indicator for the current cycle — e.g., "7 of 10 members paid" or a progress bar — with no names attached, so they still know whether the payout is on track. 4. If the Member is flagged Overdue themselves, they see that on their own "My Payments" view (it's their own status), and receive the standard reminder notifications.

Business Rules:

- Members cannot see any other individual member's name paired with a payment status, on any screen, at any time. - The full per-member ledger (names + status) is editable (mark-as-paid) and viewable only by Organizer/Co-Organizer, ensuring one source of truth without exposing individual members' financial standing to the group. - This rule also governs FR-PAYOUT-02 (overdue flags) and the Risk Flag in FR-TRUST-02 — both are Organizer/Co-Organizer-only by the same logic.

3.8 Trust Score

FR-TRUST-01 Portable Trust Score

The system shall calculate and display a transparent Trust Score for every user, based on their on-time payment rate and completed-committee history across all committees they have participated in.

Actors: All Users *Preconditions:* User has participated in at least one committee cycle.

Main Flow:

1. System tracks each Member's on-time payment rate and number of successfully completed committees. 2. Score is calculated using a simple, disclosed formula (not a black box) and displayed on the user's profile. 3. Other Organizers can view a prospective Member's Trust Score before approving their join request.

Business Rules:

- Trust Score is portable across committees and persists with the user account, not with any single committee. - Trust Score is a summary reputation number only — it does not reveal which specific committees or cycles a payment was late in to anyone other than the user themselves and platform Admin.

FR-TRUST-02 Predictive Payment Risk Flag

Ahead of each cycle's due date, the system shall flag Members who show a pattern suggesting they may miss this cycle's payment, so the Organizer/Co-Organizer can follow up personally before the deadline instead of finding out after.

Actors: Organizer, Co-Organizer, System *Preconditions:* Member has at least one prior cycle's payment history, in this or another committee.

Main Flow:

1. A few days before each cycle's due date, the system evaluates each Member's payment pattern: frequency of late payments, a worsening lateness trend, and repeated "just-in-time" payments (paid within hours of the deadline), pulling from cross-committee history feeding the Trust Score. 2. Members scoring above a defined risk threshold are marked "At Risk" for that upcoming cycle. 3. The Organizer/Co-Organizer sees an "At Risk" badge next to that Member's name on their (private, full) ledger, with a one-tap "Send personal reminder" action. 4. The flag clears automatically once the Member pays for that cycle, or the cycle closes.

Alternate / Exception Flows:

- Member has insufficient history (e.g., first cycle ever) → no flag is shown; the Member defaults to neutral rather than being guessed at.

Business Rules:

- The Risk Flag is visible only to the Organizer/Co-Organizer of that specific committee — never to other Members, and never shown on the Member's public profile. It is an operational nudge, not a public reputation mark. - This ships as a simple, disclosed rules-based heuristic (e.g., "2+ late payments in the last 3 cycles" or "consistently pays within the final hours of the deadline"). A Gemini-reasoned version using richer context is a natural Phase 2 upgrade, not required for MVP. - This extends, but does not replace, FR-TRUST-01: Trust Score is retrospective and portable across the platform; the Risk Flag is predictive, per-cycle, and scoped to one committee's organizers.

3.9 Notifications & Reminders

FR-NOTIF-01 Automated Reminders & Alerts

The system shall send automated notifications for payment due dates, approvals, payouts, and committee lifecycle events via push notification, SMS, and/or WhatsApp.

Actors: All Users *Preconditions:* User has an active account and at least one committee membership.

Main Flow:

1. Scheduled jobs check upcoming due dates and trigger reminders ahead of, on, and after the due date. 2. System sends contextual notifications for: join requests received, join request approved/rejected, payment marked received, payout released, cycle overdue flag raised (to Organizer/Co-Organizer and the overdue Member only), new complaint status update, and proactive Committee Health recommendations (FR-MONITOR-01).

Business Rules:

- Users can manage notification channel preferences (push / SMS / WhatsApp) from their profile settings.

3.10 Chatbot (FAQ & How-To)

FR-BOT-01 In-App FAQ Assistant

The system shall provide a chatbot, powered by the Google Gemini API (free tier), that answers frequently asked questions and "how do I…" style guidance about using Sanjhi.

Actors: All Users *Preconditions:* User has access to the Support section of the app.

Main Flow:

1. User opens Support and selects the Chatbot option. 2. User types a question (e.g., "How do I change my payment mode?"). 3. System sends the query to the Gemini API with app-specific context/prompting and displays the response.

Alternate / Exception Flows:

- Query is outside the chatbot's scope (e.g., a specific account dispute) → chatbot suggests filing a complaint or contacting support instead of attempting to resolve it.

Business Rules:

- Chatbot scope is intentionally limited to FAQ/how-to guidance, both to stay within Gemini's free-tier rate limits and to avoid the chatbot being relied on for account-specific or financial-dispute matters.

3.11 Customer Support & Complaints

FR-SUPPORT-01 File a Complaint or Report

Any user shall be able to file a complaint or report against another user or a committee, which is routed to the Admin queue for review.

Actors: Organizer, Co-Organizer, Member *Preconditions:* User is logged in.

Main Flow:

1. User opens Support and selects "File a Complaint / Report." 2. User selects a category (e.g., payment dispute, harassment, suspected fraud, other), the related committee/user, and a description; may attach evidence (e.g., a screenshot). 3. User submits the complaint; it appears in the Admin's complaint queue with a Pending status. 4. Admin reviews the complaint (with access to the relevant committee's ledger for context), takes action, and updates the status. 5. Complainant is notified of the outcome.

Business Rules:

- Complaints are visible only to the filer and Admin — not to the accused party or other committee members — to prevent retaliation while under review.

FR-SUPPORT-02 Complaint Case-Builder Agent

When a complaint is submitted, the system shall run an investigative agent that gathers evidence, cross-checks the complainant's claimed timeline against the actual payment record, and produces a structured Case File with a recommended action — giving Admin a defensible evidentiary picture to open every complaint with, rather than raw text plus a guessed priority label.

Actors: Admin, System (Case-Builder Agent), Gemini API, Ledger & Trust Score / Risk Flag data (via tool call) *Preconditions:* A complaint has been submitted (FR-SUPPORT-01).

Main Flow:

1. On submission, the agent pulls the complaint's full text — category, description, and any attached evidence — and identifies the two parties involved (complainant and accused). 2. Tool call: the agent pulls the relevant Ledger entries and Risk Flag / Trust Score history for both parties, scoped to the one committee the complaint concerns. 3. The agent cross-checks the complainant's claimed timeline (dates/amounts described in the complaint text) against the actual payment timestamps recorded in the Ledger. 4. The agent flags any contradiction in plain language with the specific evidence behind it — e.g., "member claims they paid on the 3rd, but no submission exists until the 5th" — or, if none is found, notes the timeline as consistent. 5. The agent compiles a Case File: a plain-language narrative summary, the full evidence trail (the specific ledger entries and timestamps referenced), the contradiction findings (or consistency note), and a recommended action with a suggested priority. 6. Admin opens the complaint queue and sees the Case File alongside the original submission and evidence — not a one-line summary. 7. Admin accepts, edits, or overrides any part of the Case File, then proceeds with resolution as in FR-ADMIN-01 / FR-SUPPORT-01.

Alternate / Exception Flows:

- Gemini API is unavailable or rate-limited → the complaint still appears in the queue with its raw content and the pulled ledger data attached, but without the synthesized narrative; Admin is never blocked from acting. - One or both parties have no prior ledger/trust history in the relevant committee → the agent states "insufficient history to cross-check" rather than guessing. - Evidence is genuinely ambiguous (e.g., a partial or illegible screenshot) → the agent surfaces the ambiguity explicitly instead of resolving it silently.

Business Rules:

- The agent is advisory only — it never auto-resolves, auto-suspends, auto-dismisses, or auto-prioritizes a complaint without Admin sign-off; every Case File requires human review before action is taken. - The agent's tool calls are scoped to exactly the two parties and the one committee involved in the complaint — never any unrelated member's data — consistent with FR-SUPPORT-01's confidentiality rule and enforced at the API level (see Section 4.2). - Case Files are visible only to Admin, never to the complainant or accused party, consistent with FR-SUPPORT-01. - Every tool call and the resulting Case File are retained with the complaint record, so Admin's eventual decision is traceable back to the exact evidence the agent examined. - Why it matters: a defensible, evidence-backed Case File is a far stronger basis for a dispute-resolution decision than raw complaint text and a guessed priority label.

3.12 Admin Panel

FR-ADMIN-01 Platform Administration

The system shall provide an Admin role with tools to oversee all users and committees, resolve complaints, and take corrective action.

Actors: Admin *Preconditions:* Admin has a platform-level account, separate from the Organizer/Member role model.

Main Flow:

1. Admin logs into the Admin console. 2. Admin can search/view all users (profile, Trust Score, committee memberships) and all committees (settings, full ledger, member list). 3. Admin reviews the complaint queue — including the agent-built Case File from FR-SUPPORT-02 — opens individual complaints with full context, and takes action. 4. Available actions include: suspend a user account, freeze a committee, resolve/dismiss a complaint, and view platform-wide activity and Committee Health recommendations (FR-MONITOR-01).

Business Rules:

- Admin actions (suspensions, freezes, resolutions) are logged for accountability. - Admin's ability to view the full per-member ledger (unlike ordinary Members) is a platform-oversight exception to FR-LEDGER-01, required for dispute resolution, and is itself logged.

3.13 Committee Health Monitoring

Every other feature in this document reacts to something a user just did. FR-MONITOR-01 below is the exception: it runs on its own schedule, watches how a whole committee is trending over time, and speaks up before a problem becomes a crisis — Sanjhi's first feature with a persistent memory of the past.

FR-MONITOR-01 Committee Health Monitor Agent

The system shall run a background agent, not triggered by a user action, that periodically scans every active committee for early-warning patterns — rising Awaiting-Confirmation counts, capacity stalling, or a cluster of At-Risk members converging on the same cycle — and proactively surfaces a recommendation to the Organizer before it becomes a crisis, rather than leaving everything reactive.

Actors: System (Health Monitor Agent), Organizer, Scheduled Job Runner *Preconditions:* Committee is active (not closed) and has enough cycle history to establish a baseline.

Main Flow:

1. A scheduled job wakes the agent at a fixed cadence (e.g., daily, or at defined checkpoints within each cycle) — independently of any user opening the app. 2. The agent iterates over every active committee on the platform. 3. For each committee, the agent computes early-warning signals from current and historical data: a rising trend in Awaiting-Confirmation counts, capacity stalling (join requests going unfilled), and a cluster of At-Risk members (per FR-TRUST-02) converging on the same upcoming cycle. 4. The agent compares these signals against that specific committee's own historical baseline — stored in Health Memory — and configured thresholds, rather than a one-size-fits-all cutoff. 5. If an early-warning pattern is detected, the agent generates a plain-language recommendation: what's happening, which members/cycle are implicated, and a suggested next step (e.g., "3 members are trending toward missing Cycle 7 — consider reaching out before the due date"). 6. The recommendation is proactively surfaced to the Organizer — in-app and via notification (FR-NOTIF-01) — before the cycle closes and the pattern becomes an actual default. 7. If no pattern is detected for a committee, the agent logs the snapshot to Health Memory and moves on to the next committee. 8. Once the Organizer views or acts on a recommendation, the outcome is written back to Health Memory, sharpening the baseline used for that committee's future comparisons.

Alternate / Exception Flows:

- Committee has too little history to establish a baseline (e.g., its first cycle) → no recommendation is generated; the agent logs the snapshot and waits for more data, mirroring the "insufficient history" rule in FR-TRUST-02. - Multiple recommendations would fire for the same committee in a short window → they are batched into a single notification rather than repeatedly alerting the Organizer. - Organizer dismisses a recommendation → the dismissal is recorded in Health Memory so the identical pattern is not re-flagged immediately.

Business Rules:

- This is a background/proactive agent: it is never triggered by a Member or Organizer action, and it produces no output at all unless it detects a genuine early-warning pattern — silence is the default, not noise. - Health Memory is scoped per committee — one committee's history never informs another committee's baseline directly, though platform-wide default thresholds seed a brand-new committee until it has enough history of its own. - Recommendations are visible only to the Organizer/Co-Organizer of that committee — the same visibility scope as the Risk Flag in FR-TRUST-02 — and are never broadcast to Members. - The agent recommends; it never acts — it cannot extend deadlines, message members directly, or alter the ledger. All follow-up remains a manual Organizer action, consistent with Sanjhi's non-custodial, non-automated-enforcement design (Section 2.5). - This ships as a disclosed, rules-based heuristic over signals already tracked elsewhere in the product (Ledger, Risk Flag, capacity) — deliberately avoiding an extra Gemini dependency for a feature that doesn't need language generation to be useful. A Gemini-reasoned version using richer qualitative context is a natural Phase 2 upgrade, the same evolution path already planned for FR-TRUST-02 (Section 7). - Why it matters: this is the clearest "agent with memory across time" story in the product — it watches, remembers, and speaks up on its own, rather than waiting to be asked.

3.14 WhatsApp Conversational Agent (Chat-Based Task Execution)

Sanjhi's core workflows were originally coordinated informally over WhatsApp; this feature brings a structured version of that same channel back into the product itself. Rather than requiring a user to open the web or mobile app, the WhatsApp Conversational Agent lets an existing or prospective user log in or sign up, and then execute the same core account and committee actions, entirely through natural-language chat inside WhatsApp.

Architecturally, the agent is a thin conversational layer over infrastructure the product already has: an inbound/outbound WhatsApp connection, a Groq-hosted large language model performing function-calling (deciding which action the user wants and with what parameters), and a Sanjhi MCP (Model Context Protocol) tool layer that wraps the existing backend APIs — authentication, committees, payments, complaints — as a fixed set of callable tools. The agent itself never touches the database directly; every action it takes goes through the same authenticated, role-checked API the web and mobile apps already use (see Section 4.2).

FR-WABOT-01 WhatsApp Login & Sign-Up via Chat

A user messaging the Sanjhi WhatsApp number for the first time, or an unauthenticated returning user, shall be guided through a Login or Sign-Up choice and OTP verification entirely within the chat, before any account action is permitted.

Actors: New User, Returning User, WhatsApp Conversational Agent *Preconditions:* User has sent a message to the Sanjhi WhatsApp number and does not currently have an authenticated Conversational Session on that chat.

Main Flow:

1. User sends any message (e.g., "Hi") to the Sanjhi WhatsApp number. 2. Agent replies with a welcome message and two options: "Login" or "Sign Up." 3. If the user selects Login, the agent asks whether to use the WhatsApp number the user is messaging from, or a different registered number. 4. Agent sends a 6-digit OTP via WhatsApp to the selected number. 5. User replies with the OTP within the validity window. 6. On successful verification, the agent establishes an authenticated Conversational Session for that WhatsApp chat, greets the user by name, and confirms they can now ask it to perform tasks. 7. If the user instead selects Sign Up, the agent collects the minimum required details (phone number and display name) and follows the same OTP verification path as FR-AUTH-01 before creating the account.

Alternate / Exception Flows:

- OTP is invalid, expired, or not entered within the window → agent offers to resend and re-prompts, mirroring FR-AUTH-01/FR-AUTH-02's retry and cooldown rules. - User selects "Login" but the number they choose to verify has no Sanjhi account → agent informs them and offers to switch to Sign Up instead. - User tries to request an account action before completing authentication → agent re-prompts with the Login/Sign-Up choice rather than attempting the action.

Business Rules:

- OTP delivery, expiry (5 minutes), single-use, and retry-cooldown rules are identical to FR-AUTH-01 and FR-AUTH-02 — WhatsApp is a new delivery and entry channel, not a separate, weaker authentication path. - A Conversational Session maps one WhatsApp chat to exactly one authenticated Sanjhi user at a time; logging in as a different user first requires an explicit logout. - No account action (Section 3.14, FR-WABOT-02) is available to the agent until the Conversational Session is authenticated.

FR-WABOT-02 Conversational Task Execution via MCP Tool-Calling

Once authenticated, a user shall be able to ask the WhatsApp Conversational Agent, in plain language, to perform any core account or committee action already available to them in the app, and receive a natural-language response summarizing the result.

Actors: Authenticated User (Organizer, Co-Organizer, or Member), WhatsApp Conversational Agent, Groq API, Sanjhi MCP Tool Layer *Preconditions:* User has an authenticated Conversational Session (FR-WABOT-01).

Main Flow:

1. User sends a free-text request (e.g., "what are my committees," "pay this month's contribution," "create a new committee for 10 people," or "file a complaint about a late payment"). 2. The Groq-hosted agent interprets the request and selects the matching MCP tool (e.g., get_my_committees, make_payment, create_committee, join_committee, file_complaint), extracting any needed parameters from the message or asking a short follow-up question if a required parameter is missing. 3. The MCP tool layer calls the corresponding Sanjhi backend API using the authenticated user's own session credentials, so the action is authorized exactly as if performed in the app. 4. The backend returns the result (e.g., committee list, payment confirmation, new committee's invite code) to the tool layer. 5. The agent formats the result into a natural-language reply and sends it back in the same chat. 6. For any action that has a Review & Confirm step in the app (e.g., creating a committee, FR-CC-01), the agent restates the parsed details and asks the user to confirm in chat before the MCP tool actually executes the write.

Alternate / Exception Flows:

- Requested action is outside the user's role permissions (e.g., a Member asking to mark another member's payment as received) → the underlying API call is rejected by the same role checks as the REST API, and the agent explains that the action isn't available to their role rather than attempting a workaround. - Groq API is unavailable or rate-limited → the agent tells the user it's temporarily unable to process requests and suggests using the app directly; no request is silently dropped or guessed at. - The agent cannot confidently match the message to any available tool → it asks a clarifying question or points the user to the FAQ chatbot (FR-BOT-01) rather than guessing at an action. - A requested write action (payment, committee creation, complaint) is not confirmed by the user within the conversation → the agent takes no action and the pending request expires.

Business Rules:

- Every MCP tool call executes strictly within the authenticated user's existing role and permissions (Section 2.2) — the agent has no elevated or service-level access to any user's data beyond what that user's own session already permits. - The set of exposed tools mirrors existing, already-authorized REST endpoints only; the WhatsApp Conversational Agent introduces no new backend capability, only a new, conversational way to invoke ones that already exist. - Any confidentiality rule that applies elsewhere in this SRS (e.g., FR-LEDGER-01's ledger visibility, FR-SUPPORT-01's complaint privacy) applies identically to what the agent is permitted to read or say back in chat. - Write actions (payments, committee creation, joining, complaints) require an explicit in-chat confirmation step before the

corresponding tool executes, mirroring the app's own Review & Confirm patterns. - The agent's default reply language is English, consistent with Sanjhi's launch localization (Section 4.5); basic Urdu message recognition may be attempted best-effort, with a full bilingual conversational experience deferred to the same Phase 2 Urdu toggle already planned for the rest of the product.

Figure 3.14-1 — WhatsApp Conversational Agent flow: chat message → Groq function-calling agent → Sanjhi MCP tool layer → existing backend API → natural-language reply.

4. Non-Functional Requirements

4.1 Performance

- Ledger updates shall reflect a marked payment within 2 seconds for all authorized viewers.
- OTP delivery shall complete within 30 seconds under normal SMS/email gateway conditions, including OTPs delivered via the WhatsApp Conversational Agent (FR-WABOT-01).
- The Complaint Case-Builder Agent shall complete its tool-call-and-cross-check pipeline within a bounded time (target 15 seconds) before falling back to displaying raw complaint content, so Admin is never left waiting on the agent.
- The Committee Health Monitor Agent's scan across a committee's full portfolio shall complete within its scheduled window without delaying reminder or cycle-rollover jobs that share the same queue.
- The WhatsApp Conversational Agent shall respond to a user's message, including the underlying MCP tool call, within a target of 5 seconds under normal conditions, so the chat feels responsive rather than app-like in latency.

4.2 Security & Privacy

- All traffic shall be encrypted in transit (HTTPS/TLS).
- OTP codes shall expire after 5 minutes and be single-use.
- Session tokens shall be revocable (e.g., on logout from all devices) and shall use a refresh-token pattern rather than indefinitely-lived access tokens.
- Sensitive financial fields (linked account numbers) shall be masked in the UI except where the Organizer or paying Member specifically needs to view them in full.
- Per-member payment status is access-controlled by role at the API level, not just hidden in the UI — a Member's client shall never receive other members' individual status data, so the restriction can't be bypassed by inspecting network traffic.
- Agent tool calls (Complaint Case-Builder, Committee Health Monitor, and the WhatsApp Conversational Agent's MCP tool calls) are scoped server-side to only the data and actions the calling context is authorized to see or perform — enforced at the API level, not by prompt instructions alone — mirroring the confidentiality rules in FR-SUPPORT-02 and FR-MONITOR-01.
- WhatsApp Conversational Sessions authenticate using the same OTP-plus-refresh-token pattern as FR-AUTH-02; a WhatsApp chat is simply another authenticated client of the same backend, not a separate trust boundary. Session tokens issued to a Conversational Session are revocable the same way as any other session.
- The MCP tool layer shall never expose a tool whose underlying API call is not already gated by the requesting user's own role and JWT — there is no tool-level bypass of existing authorization checks.

4.3 Scalability & Reliability

- Scheduled reminder, cycle-rollover, and risk-flag jobs shall run on a queue-based system to handle growth in concurrent committees without missed or duplicate notifications.
- The system shall degrade gracefully if the Gemini API free-tier rate limit is hit across any of its three uses (chatbot, NL setup parsing, Complaint Case-Builder narrative synthesis) — each feature shall fail independently to a manual fallback (static FAQ page, blank form, raw complaint content without narrative) rather than blocking the user.

- The Committee Health Monitor Agent's scheduled scans shall run on the same queue-based infrastructure as reminders and cycle rollovers, scaling horizontally as the number of committees grows; a scan failure for one committee shall not block scans of any other.
- The WhatsApp Conversational Agent shall degrade gracefully if the Groq API or the WhatsApp connection itself is unavailable — the agent informs the user it is temporarily unable to help and points them to the app, rather than leaving the message unanswered or retrying silently forever.
- The Sanjhi MCP tool layer shall be stateless between calls, with all session state held in the Conversational Session store, so it can scale horizontally alongside the rest of the backend.

4.4 Usability & Accessibility

- Core flows (create, join, pay, view my payments) shall be completable within 3–4 taps/screens from the Dashboard.
- UI shall follow a warm, community-oriented "desi" visual language (Section 6.1) to feel familiar and trustworthy to the target audience, rather than clinical/corporate fintech styling.
- Agent-generated content (Case Files, Health recommendations) shall be visually distinguished from user-entered content, so it is always clear which parts of a screen came from an agent versus a human.
- The WhatsApp Conversational Agent's onboarding (Login/Sign-Up choice, OTP entry) shall be completable in no more than 4 message exchanges under normal conditions, keeping the chat-based flow at least as fast as the equivalent app screens.

4.5 Localization

- Ships in English with South Asian/desi terminology used intentionally for committee-related labels (e.g., referencing "Bisi/Kameti" alongside "Committee").
- Phase 2/3: full Urdu language toggle, allowing users to switch the entire app interface to Urdu; the WhatsApp Conversational Agent's full bilingual support follows the same Phase 2 timeline (Section 3.14).

4.6 Compliance & Data Handling

- Sanjhi does not hold, escrow, or move customer funds, which keeps it outside the scope of funds-holding financial licensing that would otherwise apply.
- Identity data collected is limited to phone number/email and OTP verification — no CNIC or government ID is stored, regardless of whether the account was created via the app or the WhatsApp Conversational Agent.
- A Member's individual payment status is treated as personal financial data and is restricted to that Member and the committee's Organizer/Co-Organizer, per FR-LEDGER-01.
- Ledger and Trust Score/Risk Flag data pulled by either agent (Case-Builder, Health Monitor) is used only in-session to build that agent's output and is retained solely as part of the resulting Case File or Health Memory record — no new categories of personal data are introduced by either agent.
- Data returned to the WhatsApp Conversational Agent via MCP tool calls is used only to compose the immediate chat reply and is not separately retained by the agent beyond normal chat history already implicit in WhatsApp itself; no new categories of personal data are introduced by this feature.

5. External Interface Requirements

5.1 Payment-Related Interfaces

Manual Mode (the only mode in this release): no payment API integration is required — the app only stores and displays the Organizer's linked account details (JazzCash / Easypaisa / bank) as reference information for Members. There is no aggregator or automatic-checkout integration in this version; see Section 7 for that as a possible future item.

5.2 Notification Interfaces

- SMS gateway for OTP delivery and text reminders.
- WhatsApp Business API for one-way reminders and key alerts, reflecting how committees already communicate today.
- Native push notifications for the mobile app.
- Committee Health Monitor recommendations (FR-MONITOR-01) are delivered through these same three channels — no new notification interface is introduced.

5.3 AI / Gemini Interface

Google Gemini API (free tier) powers three narrowly-scoped features, each with its own fallback so a rate-limit or outage on one never blocks the others:

- In-app FAQ/how-to chatbot (FR-BOT-01).
- Complaint Case-Builder Agent's narrative synthesis, layered on top of internal tool calls into Ledger and Trust Score / Risk Flag data (FR-SUPPORT-02).
- Natural-language committee setup parsing (FR-CC-02).

Note: the Committee Health Monitor Agent (FR-MONITOR-01, Section 3.13) intentionally does not call the Gemini API — it runs on a disclosed rules-based signal engine instead, so it adds background-agent behaviour without adding a fourth Gemini dependency. The WhatsApp Conversational Agent (Section 3.14) is a separate integration again, using Groq rather than Gemini — see Section 5.5.

5.4 Background / Scheduled Agent Interface

The Committee Health Monitor Agent (FR-MONITOR-01) is invoked by Sanjhi's own scheduled-job infrastructure (Section 1.4), not by an external API. Its interfaces are entirely internal to the platform:

- Job Scheduler — triggers the agent on a fixed cadence, independent of user activity.
- Health Memory store — a per-committee state table the agent reads and writes on every run: historical baselines, prior recommendations, and dismissals.
- Notification interface — reuses the same push / SMS / WhatsApp channels as FR-NOTIF-01 (Section 5.2) to surface recommendations to Organizers; no separate delivery channel is built.

5.5 WhatsApp Conversational Agent Interface

The WhatsApp Conversational Agent (Section 3.14) introduces a distinct, two-way interface from the one-way WhatsApp Business API alerts described in Section 5.2. Its components are:

- WhatsApp Messaging Connection — a persistent connection (e.g., via a WhatsApp Web-based library) used to both receive incoming user messages and send OTPs and task replies, distinct from the outbound-only WhatsApp Business API channel used for FR-NOTIF-01.
- Groq API — the LLM function-calling engine that interprets each incoming message, selects the matching MCP tool and parameters (or asks a clarifying question), and drafts the natural-language reply.
- Sanjhi MCP Tool Layer — an internal tool-calling interface, built on the existing REST APIs, exposing a fixed set of authenticated actions (e.g., login_with_otp, verify_otp, get_my_committees, create_committee, make_payment, join_committee, get_dashboard, file_complaint) that the Groq agent may invoke on behalf of the authenticated user only.
- Conversational Session Store — holds each WhatsApp chat's authentication state (unauthenticated, awaiting-OTP, authenticated) and short-term task context, following the same session-revocation rules as Section 4.2.

Note: like the Health Monitor Agent, the WhatsApp Conversational Agent's LLM dependency (Groq) is intentionally separate from the Gemini usage described in Section 5.3, so an outage or rate-limit on one provider does not affect the other.

6. Design Section — Screens & Wireflows

6.1 Visual Design Direction

Sanjhi's UI should feel warm, communal, and distinctly South Asian — deep navy/charcoal grounding neutral, warm teal/emerald and gold/mustard accents, geometric sans-serif paired with rounder celebratory type, sparing jali-inspired motifs, calm/plain-language copy around money, and rounded "circle of people" iconography.

Agent-generated content (Case Files, Health recommendations) uses a small, consistent "assisted" marker (a subtle sparkle glyph plus muted gold accent) so it's always visually clear which parts of a screen were produced by an agent versus typed by a person. The WhatsApp Conversational Agent, being text-only, applies the same principle through consistent framing phrases (e.g., always confirming an action in the same plain-language pattern before executing it) rather than a visual marker.

6.2 Information Architecture

Top-level navigation (mobile): Dashboard — Committees — Support — Profile. Admin users see a separate Admin console.

- Dashboard: entry point after login; shows all committees the user belongs to across both roles, plus Create/Join actions.
- Committees: list and detail views. Organizer/Co-Organizer see a Ledger tab (full, per-member); Members see a "My Payments" tab (own history + anonymized progress) instead. The Committee Detail screen also surfaces a dismissible Health Recommendation banner whenever the Health Monitor Agent has an active flag for that committee (Organizer/Co-Organizer only).
- Support: Chatbot and Complaint/Report entry points.
- Profile: identity info, Trust Score, notification preferences, linked account management.
- WhatsApp: no separate navigation structure — the entire interface is the single chat thread with the Sanjhi WhatsApp number, gated by the Login/Sign-Up and OTP flow in FR-WABOT-01.

6.3 Wireflows

Wireflow 1 (Onboarding & Authentication) and Wireflow 3 (Join a Committee) follow the flows already detailed in Sections 3.1 and 3.3. The flows below cover committee creation, payment collection, support and complaints, the Committee Health Monitor, and the WhatsApp Conversational Agent.

Wireflow 2 — Create a Committee (Organizer) — Manual setup path, with optional NL entry point (FR-CC-02) 1. Dashboard → tap "Create Committee." 2. Committee Setup Form: fill name / amount / capacity manually, OR tap "Describe it instead" and type/dictate a free-text description — Gemini parses it and pre-fills the same fields for review. 3. Schedule: choose interval and payout order; Duration auto-calculates. 4. Link Collection Account: JazzCash / Easypaisa / Bank. 5. Review & Confirm — user can still edit any field, whichever path filled it. 6. Committee Created — invite code + link generated. 7. Invite Members screen → share code/link. 8. Pending Join Requests queue — Organizer approves/rejects each request individually before a slot is confirmed.

Wireflow 4 — Payment Collection & Reconciliation — Manual Only Collection cycle due (reminders sent to all members) 1. Member views the Organizer's linked account details (JazzCash / Easypaisa / bank). 2. Member sends funds directly, using their own banking/wallet app. 3. Member submits sender account details in-app, for matching. 4. System attempts to auto-confirm receipt where technically possible. 5. If not auto-confirmed: payment sits "Awaiting Confirmation" until Organizer/Co-Organizer manually marks it Paid (fallback of record). 6. The paying Member's own

"My Payments" record updates immediately; the committee-wide progress count (no names) updates for everyone else. 7. Once all members are marked Paid → payout is released to that cycle's recipient (FR-PAYOUT-01).

Wireflow 5 — Support, Chatbot & Complaints (Admin path) Filing a complaint (Member/Organizer) is covered in FR-SUPPORT-01; this flow picks up from Admin's review. 1. Admin opens the complaint queue. 2. Admin opens a complaint and sees the full agent-built Case File (FR-SUPPORT-02): narrative summary, evidence trail, timeline cross-check, and contradiction findings — alongside the original submission and evidence. 3. Admin accepts the recommended action, edits it, or overrides it entirely. 4. Admin resolves the complaint (FR-ADMIN-01); complainant is notified of the outcome.

Wireflow 6 — Committee Health Monitor (background, Organizer-facing) This flow has no Member/Organizer-initiated entry point; see Figure 3.13-1 for the full agent logic. 1. A scheduled job runs the Health Monitor Agent across all active committees (FR-MONITOR-01) — no screen is open when this happens. 2. If an early-warning pattern is found for a committee, a recommendation banner appears on that committee's Detail screen and a notification is sent (FR-NOTIF-01). 3. Organizer/Co-Organizer taps the banner to see the detail — which members/cycle, why it was flagged, and a suggested action. 4. Organizer acts (e.g., sends a personal reminder) or dismisses the recommendation; either outcome is written back to Health Memory.

Wireflow 7 — WhatsApp Bot Onboarding & Task Execution Entirely chat-based; no app screens are involved (FR-WABOT-01, FR-WABOT-02). 1. User messages the Sanjhi WhatsApp number for the first time → agent replies with Login / Sign Up options. 2. User selects Login → agent asks whether to verify the current WhatsApp number or a different registered number. 3. Agent sends an OTP over WhatsApp; user replies with the code. 4. Agent verifies the OTP, establishes an authenticated Conversational Session, and greets the user by name. 5. User sends a free-text task request (e.g., "check my committees," "pay this month," "create a committee"). 6. Groq agent selects the matching MCP tool, calls the backend under the user's own authenticated session, and replies with the result in plain language. 7. For write actions (payment, committee creation, join, complaint), the agent restates the details and asks for a chat confirmation before the tool executes.

Figure 6.3-1 — WhatsApp Bot Onboarding & Task Execution: from first message to an authenticated, tool-executing chat session.

6.4 Screen-by-Screen Detail (key screens)

Screens not listed below (Splash, Sign Up/Login, OTP, Profile Setup, Dashboard, Notifications Center, Support Home, Chatbot, My Complaints, User Management, Committee Oversight) follow the standard visual language in Section 6.1. The screens most relevant to the flows above are detailed here.

Committee Setup Form (Organizer) — Adds an optional "Describe it instead" text/voice field above the manual fields (name, amount per member, capacity), with a "Parse" action that pre-fills the same fields (FR-CC-02). Manual entry remains fully available and is the default if the field is left blank.

Committee Detail (Organizer / Co-Organizer view) — Ledger tab shows the full per-member list with status chips (Paid / Awaiting Confirmation / Overdue), with an "At Risk" badge (FR-TRUST-02) next to members likely to miss the current cycle, and a one-tap "Send personal reminder" action on that badge. A dismissible Health Recommendation banner (FR-MONITOR-01) appears above the Ledger tab whenever the Health Monitor Agent has flagged this committee.

Committee Detail (Member view) — Tabs are Ledger-progress (aggregate only, no names), Members (names/roles only, no payment status), and My Payments (the Member's own history). No Settings or Invite-management tabs. "Pay Now" shows only the Manual flow — no payment-mode toggle, since Automatic mode does not exist in this release.

My Payments (Member) — Shows the Member's own status and history across cycles and committees, their own next due date/amount, and their own upcoming payout turn. Does not show any other member's name or status.

Pay Now (Member) — Organizer's linked account details with copy-to-clipboard, and a "Submit Sender Details" form. Manual is the only path — there is no payment-mode toggle.

Complaint Queue (Admin) — Each complaint shows the agent-built Case File (FR-SUPPORT-02) — narrative summary, evidence trail, timeline cross-check with any contradictions highlighted, and a recommended action — alongside the original submission, evidence, and linked committee context. Admin can accept the suggestion or override it before resolving.

Complaint Case File (Admin) — Full-detail expansion of a complaint: the narrative summary at the top, a side-by-side timeline comparison (member's claimed dates vs. actual Ledger timestamps, with contradictions highlighted in red), the linked evidence and ledger entries the agent cited, and an Accept / Edit / Override control on the recommended action before Admin resolves the complaint.

Committee Health Banner (Organizer / Co-Organizer) — A dismissible banner on the Committee Detail screen, surfaced only when FR-MONITOR-01 has an active recommendation for that committee. Expands to show which members/cycle triggered it, the reasoning in plain language, and a suggested action (e.g., a one-tap "Send reminder" shortcut); dismissing it records the outcome back to Health Memory.

WhatsApp Conversational Agent (Chat Thread) — Not a native-app screen — the "interface" is the WhatsApp chat thread itself with the Sanjhi WhatsApp number (FR-WABOT-01, FR-WABOT-02). The agent's own messages consistently restate any write action in plain language before asking the user to confirm, functioning as the chat-based equivalent of the app's Review & Confirm step.

7. Future Enhancements (Phase 2 / Phase 3) — Original Roadmap

- Automatic payment mode via a Pakistani payment aggregator (e.g., Rapid Gateway, Safepay) — undated; deliberately out of scope for this release; would need its own KYC/compliance workstream if picked up.
- Lottery-based and bidding-based payout order options, in addition to Fixed order.
- Optional Default Protection Pool — a small insurance contribution to cover a missed payment without disrupting the payout schedule.
- Full Urdu language toggle for the entire app interface, extended to a fully bilingual WhatsApp Conversational Agent (Section 3.14).
- Gemini-reasoned (rather than rules-based) version of the Predictive Payment Risk Flag (FR-TRUST-02), using richer behavioral context.
- Gemini-reasoned (rather than rules-based) version of the Committee Health Monitor (FR-MONITOR-01), using richer qualitative context — the same upgrade path already planned for the Risk Flag above.
- Organizer-approved automated follow-up actions the Health Monitor could trigger on its own (e.g., sending an extra reminder) — deliberately left fully manual for now, to preserve Sanjhi's non-custodial, non-automated-enforcement design.
- Advanced Trust Score visibility and tiering (e.g., unlocking larger/longer committees for highly reliable users).
- B2B white-labeled version for company-run employee welfare committees.
- Formalizing the Sanjhi MCP tool layer (Section 3.14) as a standards-compliant MCP server, so third-party AI clients beyond the WhatsApp Conversational Agent (e.g., Claude Desktop or similar tool-calling assistants) could integrate with a user's Sanjhi account under the same authenticated, role-scoped model.

8. Original Open Assumptions Log

Area	Decision
Fund custody	Sanjhi does not hold escrow; tracks the Organizer's own linked account.
Identity verification	Phone/email + OTP only; no CNIC collection.
Invite link lifetime	24 hours, regenerable on demand.
Join approval	Always required, except when the Organizer adds a user directly by User ID.
Co-Organizer rights	Equal to Organizer operationally, capped below ownership (cannot remove/replace founding Organizer).
Committee duration	Auto-calculated as Capacity × Interval; not independently editable (Fixed order only).
Missed payments	Flagged to Organizer/Co-Organizer and the specific overdue Member only; resolution left to them, not automated.
Ledger visibility	Members see only their own payment history plus an anonymized committee-wide progress count. Full per-member ledger is Organizer/Co-Organizer only.
Payment mode	Manual only in this release. Automatic/aggregator mode is a possible future item (Section 7).
Manual-mode auto-detection	Not built (SMS-parsing approach rejected due to platform policy and privacy concerns); Organizer's manual mark-as-paid is the fallback of record.
AI feature scope	Gemini used for the FAQ chatbot, the Complaint Case-Builder Agent's narrative synthesis (advisory only, never auto-decides), and NL committee-setup parsing (pre-fill only, never auto-submits).
Chatbot scope	FAQ / how-to only, to fit Gemini free-tier limits and avoid dispute-handling by the bot.
Localization	English at launch; Urdu toggle planned for Phase 2/3.
Complaint investigation scope	The Case-Builder Agent's tool calls and cross-checks are limited to the two parties and the one committee named in the complaint; it never expands to unrelated members' data.
Health Monitor cadence	Runs on a fixed schedule (exact cadence — daily vs. per-cycle checkpoint — to be finalized in engineering); recorded here for traceability.
Health Monitor reasoning approach	Ships as a disclosed, rules-based heuristic over existing signals rather than a Gemini call, avoiding an added AI dependency where narrative generation isn't required. Gemini-reasoned version deferred to Phase 2 (Section 7).
WhatsApp Bot scope	Full parity with core app actions (check committees, create committee, make payment, join committee, file complaint) exposed via MCP tool calls, strictly scoped to the authenticated user's own existing role permissions — no new capability is introduced, only a new conversational entry point.
WhatsApp Bot authentication	Login/Sign-Up choice presented in chat; Login supports verifying either the current WhatsApp number or a different registered number; OTP delivered over WhatsApp follows the same expiry/single-use/retry rules as FR-AUTH-01/02.
WhatsApp Bot AI provider	Uses the Groq API for function-calling/orchestration, kept intentionally separate from the Gemini usage powering the FAQ chatbot, NL setup parsing, and Case-Builder Agent, so an outage or rate-limit on one provider never affects the other.

Area	Decision
WhatsApp Bot language	Replies in English at launch, consistent with the rest of the product's localization; full bilingual (Urdu) conversational support is deferred to the same Phase 2 Urdu toggle planned elsewhere (Section 7).

PART II — NEW IN VERSION 2.0

Production Gap Analysis & Complete System Architecture

What actually shipped versus the original specification — and the full set of updated architecture diagrams for the platform as built.

9. Gap Analysis & Production Implementation Overview

9.1 Executive Summary & SRS Alignment Overview

The original Software Requirements Specification (Sections 1–8 of this document) established the functional baseline for digitizing traditional South Asian Rotating Savings and Credit Associations (**ROSCAs** / **Kameti / کمیٹی**).

While the initial SRS focused primarily on basic CRUD committee creation, manual payment tracking, and static roles, the **actual production implementation of Sanjhi** has evolved into a resilient financial platform featuring:

- **Multi-Agent Autonomous AI Dispute Triage** (Investigator Agent + Judge Agent with zero-hallucination tools).
- **Bilingual WhatsApp Voice Note Processing** in Roman Urdu and Urdu script using Groq Whisper Large v3 and LLaMA 3.3 70B.
- **Dual Pakistani Mobile Wallet Deep-Linking** (JazzCash and EasyPaisa).
- **0ms Zero-Latency Offline-First Caching Architecture** (PWA Service Worker + Capacitor Native Android APK).

The remainder of this document (Sections 9–14) audits the implementation against the original SRS baseline, itemizes what shipped beyond the original specification, records what remains on the roadmap, and provides a complete set of updated system architecture diagrams reflecting the production system as built.

9.2 SRS Feature Comparison & Gap Matrix

Functional Module	Baseline SRS Requirement	Actual Implemented State	SRS Gap / Extension Status
Authentication (FR-AUTH)	OTP-only mobile login	Dual phone/email auth + 6-digit BCrypt hashed OTP + Contact linking	**Exceeds SRS** (Supports secondary email/phone additions with 2-step verification)
Identity Verification	Mentioned CNIC capture	Two-sided NADRA-compliant CNIC upload + Admin approval modal + Rejection feedback	**Fully Implemented**
Committee Management (FR-COMM)	Manual committee setup form	Natural Language text & voice parsing + Instant invite codes + Public/Private marketplace toggle	**Exceeds SRS** (AI extracts schedule, turns, and dues directly from spoken voice notes)
Financial Ledger (FR-PAY)	Text ledger tracking	Double-entry cycle reconciliation + Turn tracking + Receipt proof uploads	**Fully Implemented**
Quick Payments	Generic bank transfer note	Dual JazzCash (`com.techlogix.mobilinkcustomer`) & EasyPaisa (`pk.com.telenor.phoenix`) deep-linking with auto clipboard copy	**Feature Added Beyond SRS**
Trust Scoring	Static reputation rating	Dynamic 0–1000 mathematical formula computed live over PostgreSQL ledger history	**Feature Added Beyond SRS**
Dispute Resolution (FR-DISP)	Manual admin support inbox	Autonomous 2-Agent Court (Investigator + Judge) + Auto-verdict + Human escalation queue	**Feature Added Beyond SRS**

Functional Module	Baseline SRS Requirement	Actual Implemented State	SRS Gap / Extension Status
AI Assistant (FR-AI)	Basic FAQ chatbot	Multilingual conversational voice & text assistant on Web & WhatsApp via Baileys	**Exceeds SRS**
Client Platforms	Web responsive	Hybrid: Next-gen React 19 PWA + 0ms Flash-bundled native Android APK via Capacitor 8	**Exceeds SRS**

9.3 Features Implemented Beyond Original SRS Specifications

1. Autonomous AI Dispute Resolution Court

SRS Limitation:

The original SRS (FR-SUPPORT-01/02) specified a single investigative Case-Builder Agent producing an advisory Case File for a human Admin to act on.

Production Implementation: Built out into an autonomous two-agent dispute system: - **Investigator Agent** — uses read-only database tools to inspect ledger timestamps, payment proofs, member histories, and CNIC statuses. - **Judge Agent** — evaluates the Investigator's findings against predefined bylaws and issues a verdict with a confidence score. High-confidence, low-ambiguity cases are resolved automatically in real time; complex or low-confidence cases are escalated to Super Admins, preserving the SRS's "advisory-only for hard cases" principle while removing manual triage overhead for the easy cases.

2. WhatsApp Conversational Urdu Voice Engine

SRS Limitation:

Section 3.14 specified a text-based, English-first WhatsApp Conversational Agent using Groq for function-calling, with Urdu deferred to Phase 2.

Production Implementation: Integrated `@whiskeysockets/baileys` with `fluent-ffmpeg` and `groq-sdk` to bring bilingual voice ahead of schedule: - Transcribes WhatsApp voice notes (`.ogg` / `.opus`) using **Groq Whisper Large v3**. - Understands Roman Urdu (e.g., *"Meri agli kameti kab hai?"*), Nastaliq Urdu script, and English. - Executes transactions, checks balances, and generates ICS calendar schedules directly in chat.

3. Mathematical Trust Score Model (0–1000)

SRS Limitation:

FR-TRUST-01 called for "a simple, disclosed formula" without specifying its structure or scale.

Production Implementation:

Implemented a transparent, deterministic algorithm:

Trust Score = Base Points (500) + Reliability Points + Completion Bonus + CNIC Bonus − Default/Penalty Points

Score automatically assigns tiers: **Diamond (900–1000)**, **Gold (750–899)**, **Silver (600–749)**, and **Bronze (<600)** — extending FR-TRUST-01's disclosed-formula requirement into a concrete, auditable scale.

4. 0ms Zero-Latency Offline-First Architecture

SRS Limitation:

Section 2.3 assumed constant connectivity via evergreen browsers and React Native, with no explicit offline strategy.

Production Implementation: Added a custom **Service Worker** (`public/sw.js`) and Capacitor asset bundling so all UI vector icons, styles, fonts, and scripts are stored in the phone's physical flash storage — a resilience layer not called for in the original NFRs (Section 4.3) but consistent with their spirit of graceful degradation.

9.4 Missing / Future Scope Features (Omitted in Baseline SRS)

The following high-value enterprise features were not covered in the baseline SRS (Section 7's roadmap) and represent the next development horizon, organized by theme:

- **Financial Infrastructure** — Direct 1-Link banking switch integration; automated in-app escrow wallets; split payouts & micro-loans.
- **Mechanism Design** — Auction/bidding turn ordering; lottery/blind lucky-draw turn shuffling; flexible emergency turn swapping. *(Note: Lottery and Bidding payout order already appear as Phase 2 items in the original Section 7 — this reaffirms them and adds emergency turn swapping as a new item.)*
- **Legal & Compliance** — SECP digital lending compliance; digital e-signatures for P2P promissory notes; NADRA Verisys live API biometric verification.
- **Cross-Border Remittances** — Overseas Pakistani ROSCA corridors (GCC / UK); multi-currency real-time exchange (AED/SAR/PKR).

10. Comprehensive System Architecture — High-Level Context

10.1 C4 System Context Diagram

The diagram below places the production Sanjhi platform in its full external-system context — participants, the platform itself, and every third-party dependency (WhatsApp Cloud, Groq AI, mobile wallets, SMTP).

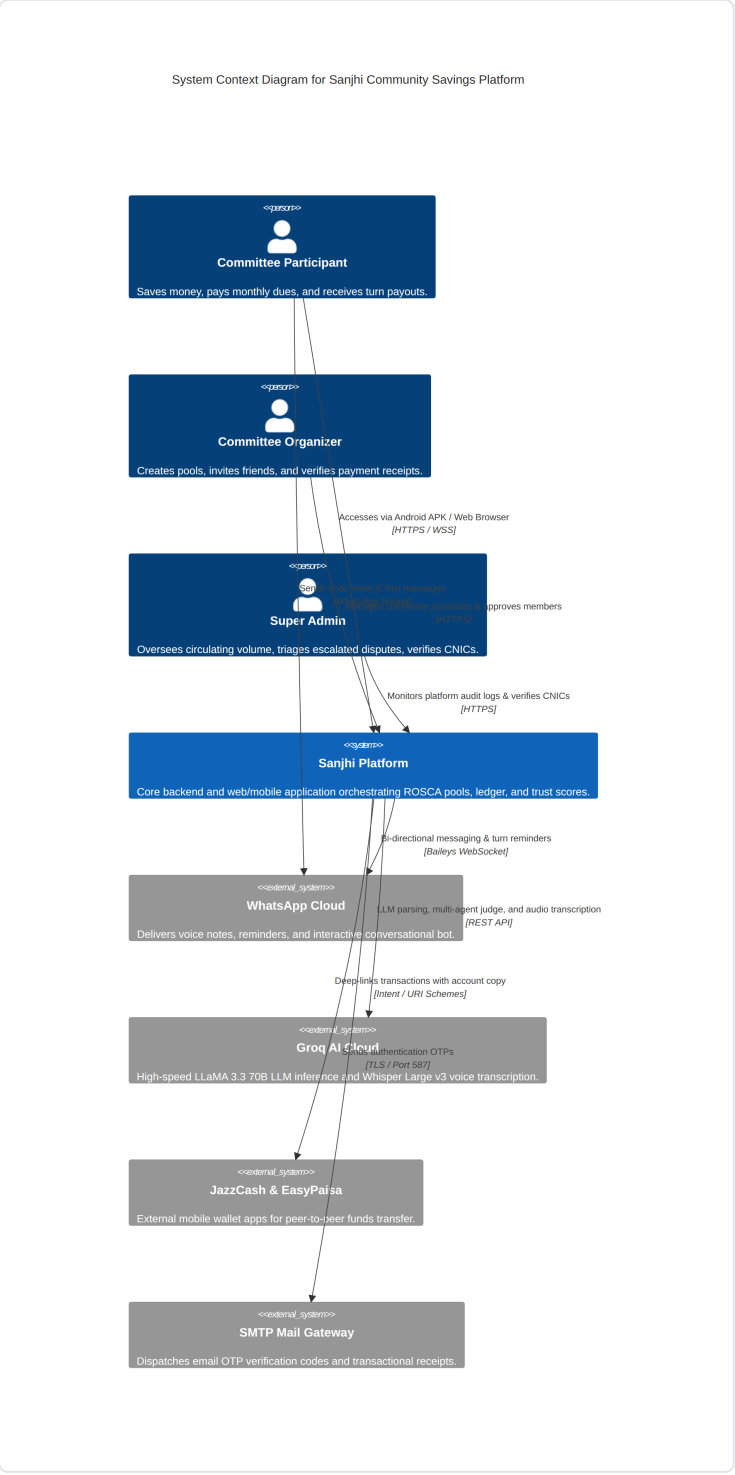

Figure 10.1 — System Context Diagram for the Sanjhi Community Savings Platform, showing all actor and external-system relationships.

10.2 Layered Modular Software Architecture

The production codebase is organized into four layers: a client Presentation Layer (React 19 SPA, Capacitor native bridge, PWA cache), an API & Network Layer (Express, JWT middleware, rate limiting, file upload handling), a Domain Services layer housing the core business logic and autonomous AI agents, and a Data & Persistence Layer.

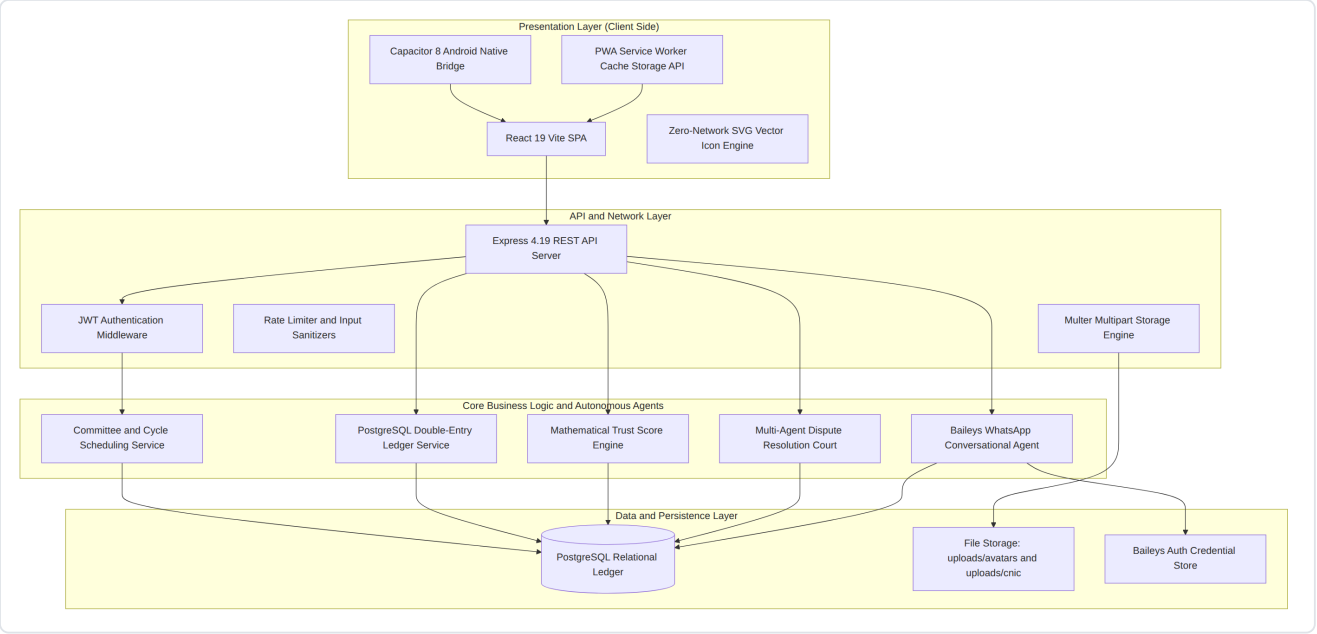

Figure 10.2 — Layered architecture from client presentation through domain services to persistence, including the Multi-Agent Dispute Court and the Baileys WhatsApp Conversational Agent as first-class domain services.

11. Database Design

11.1 Entity-Relationship Diagram (ERD)

Database schema design was explicitly out of scope for the original SRS (Section 1.2). The production schema below documents the relational model actually implemented, covering users, committees, cycles, payments, membership, complaints, and trust-score history.

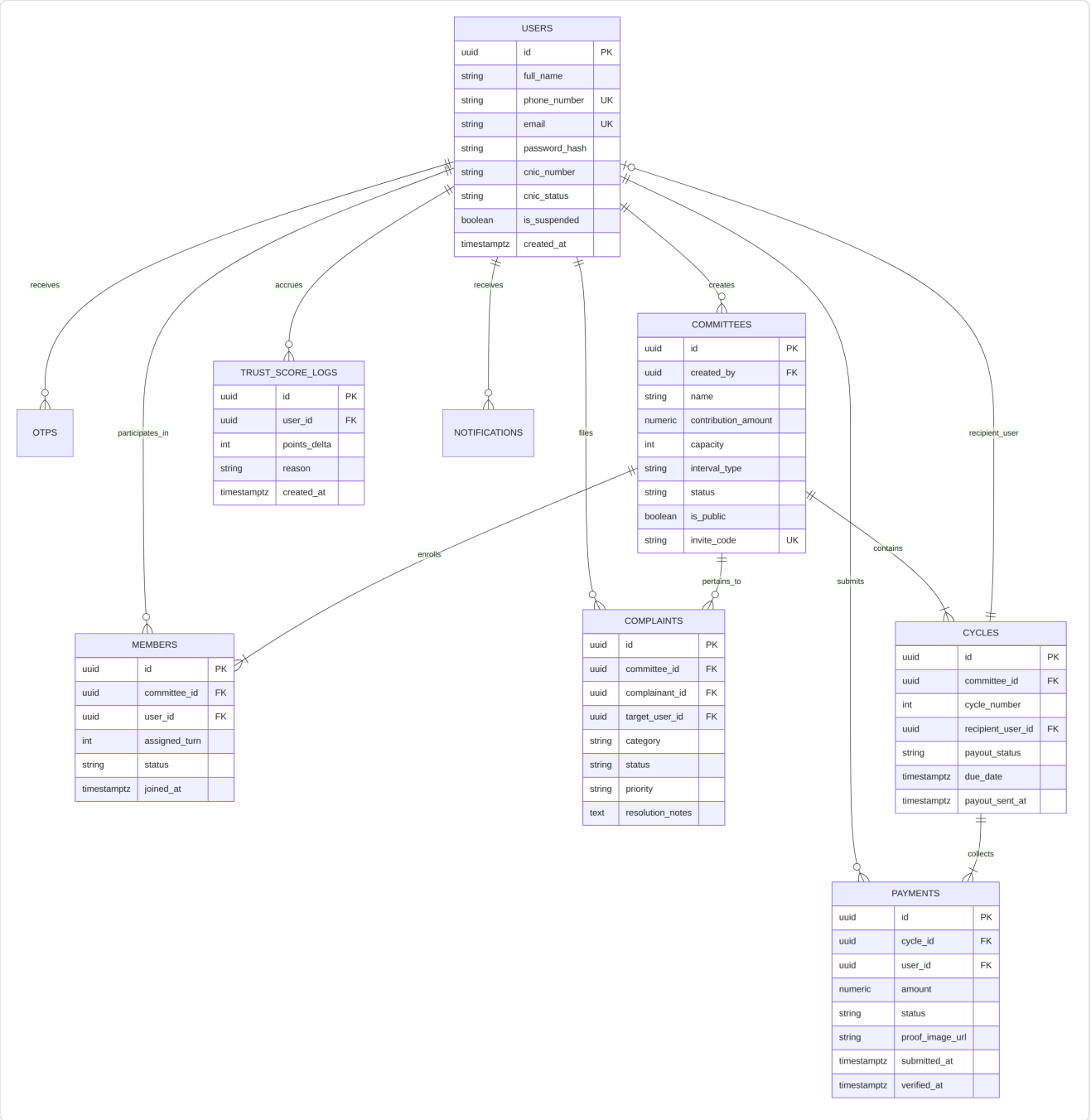

Figure 11.1 — Core relational schema: USERS as the central entity, fanning out to COMMITTEES, MEMBERS, CYCLES, PAYMENTS, COMPLAINTS, TRUST_SCORE_LOGS, OTPS, and NOTIFICATIONS.

12. AI Agent Architecture

12.1 Autonomous Multi-Agent AI Dispute Resolution Court

This sequence diagram traces a dispute from submission through the Investigator/Judge agent pipeline to either automated resolution or human escalation — the production evolution of FR-SUPPORT-02's single Case-Builder Agent.

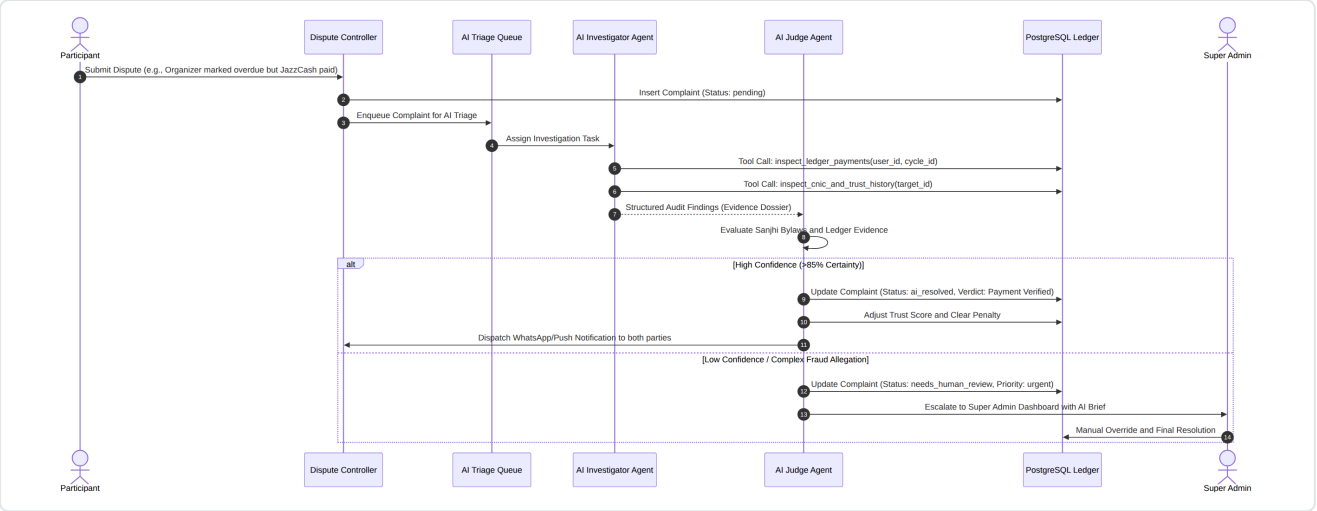

Figure 12.1 — Sequence diagram of the Investigator Agent gathering evidence, the Judge Agent evaluating it against Sanjhi bylaws, and either an automated high-confidence verdict or an escalation to Super Admin.

12.2 Multilingual WhatsApp Voice & Text Processing Pipeline

This flow shows how an inbound WhatsApp voice note becomes a completed backend action: audio conversion, Whisper transcription, intent classification, tool routing, and a formatted conversational reply — the production build-out of FR-WABOT-02's MCP tool-calling flow, extended with voice.

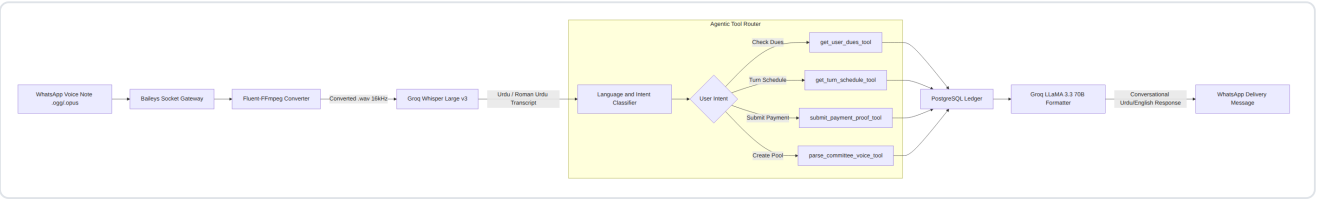

Figure 12.2 — Voice note → Baileys gateway → FFmpeg conversion → Groq Whisper Large v3 transcription → intent classification → agentic tool router → PostgreSQL → Groq LLaMA 3.3 70B response formatting → WhatsApp delivery.

13. Financial & Trust State Models

13.1 Mathematical Trust Scoring (0–1000) State Machine

This state machine documents how a user's Trust Score evolves from account creation through CNIC verification, tier progression (Silver → Gold → Diamond), risk states, and gradual restoration after a default — the concrete implementation of FR-TRUST-01 and FR-TRUST-02.

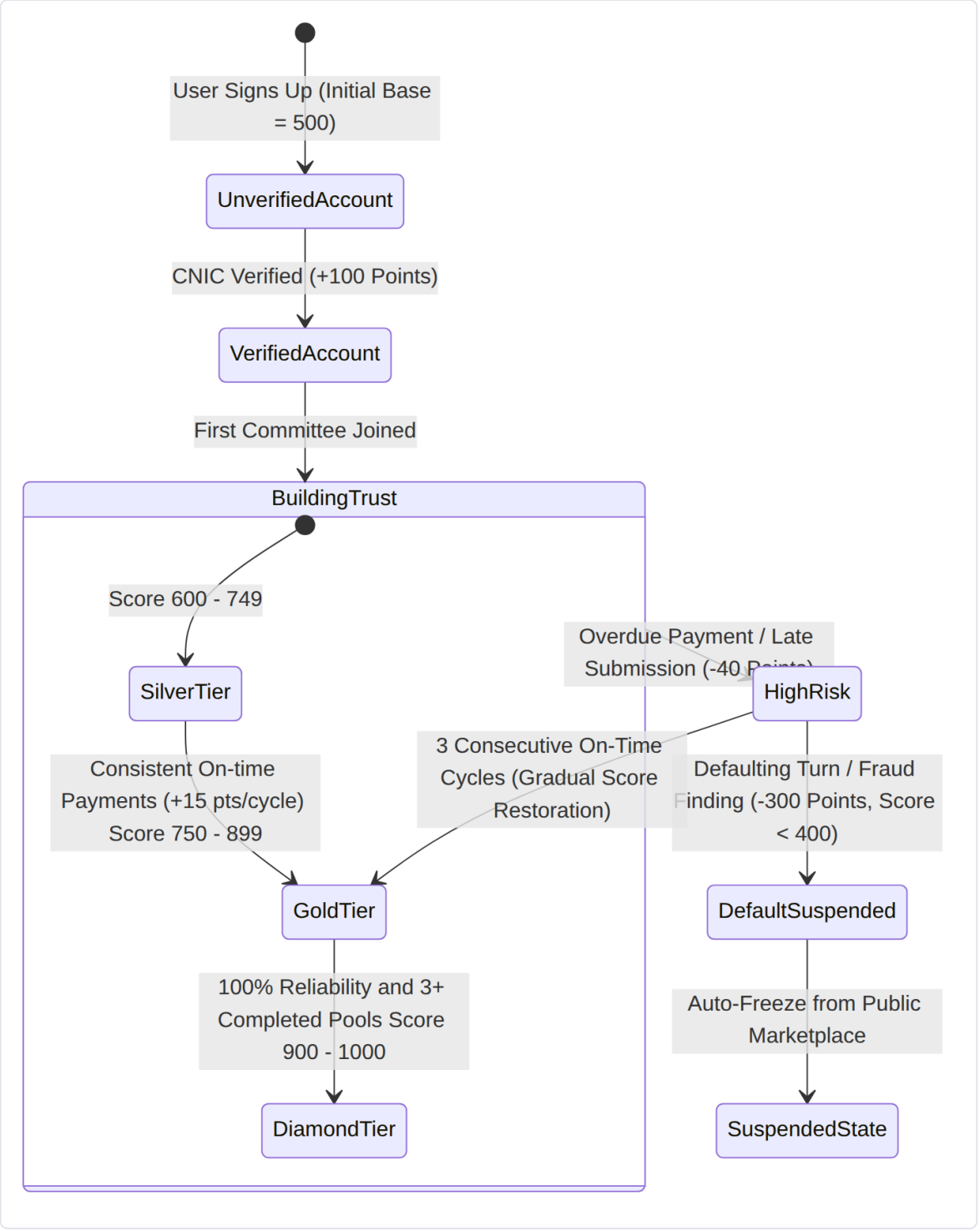

Figure 13.1 — Trust Score lifecycle: base score, verification bonus, tier thresholds, high-risk transitions, suspension, and gradual restoration.

13.2 Financial Cycle & Turn Payout State Transition

This state machine documents the lifecycle of a single committee cycle, from collection through payout confirmation and rollover to the next cycle — the production implementation of FR-PAY-01, FR-PAYOUT-01, and FR-PAYOUT-02.

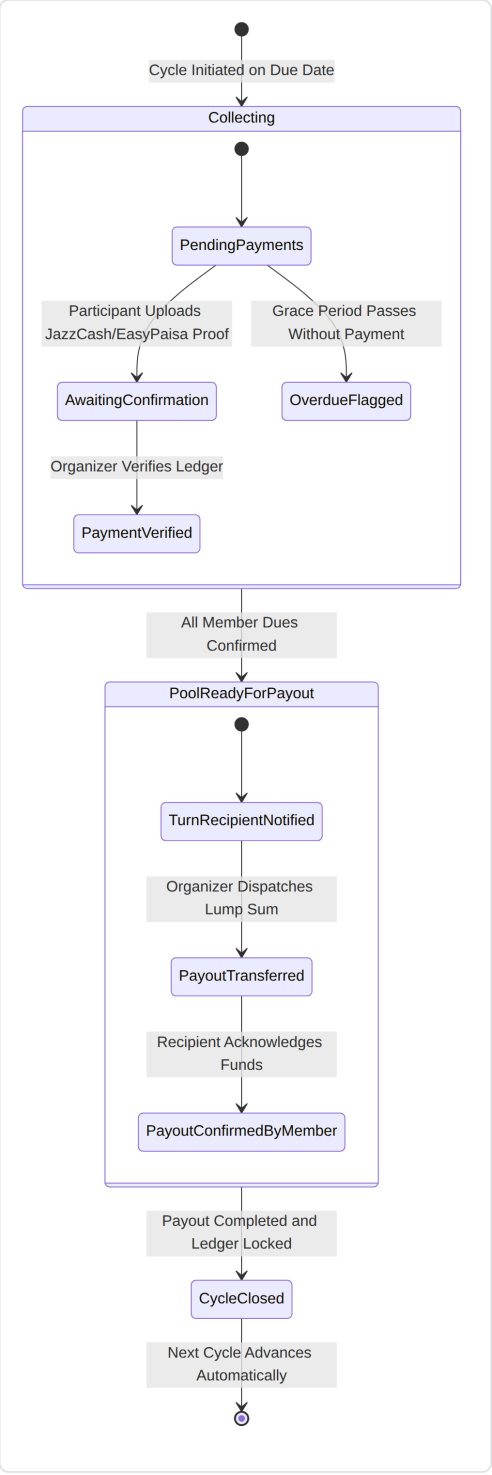

Figure 13.2 — Cycle lifecycle: Collecting (Pending → Awaiting Confirmation → Verified, or Overdue) → Pool Ready for Payout → Cycle Closed → next cycle.

14. Client Performance Architecture

14.1 Zero-Latency Mobile Storage & Service Worker Cache Engine

This flow documents the offline-first caching strategy: a cache-first approach for static assets and a network-first approach with stale-cache fallback for live ledger data, plus the Capacitor native-APK asset bundling path — extending the SRS's Section 4.3 reliability requirements with an explicit offline architecture.

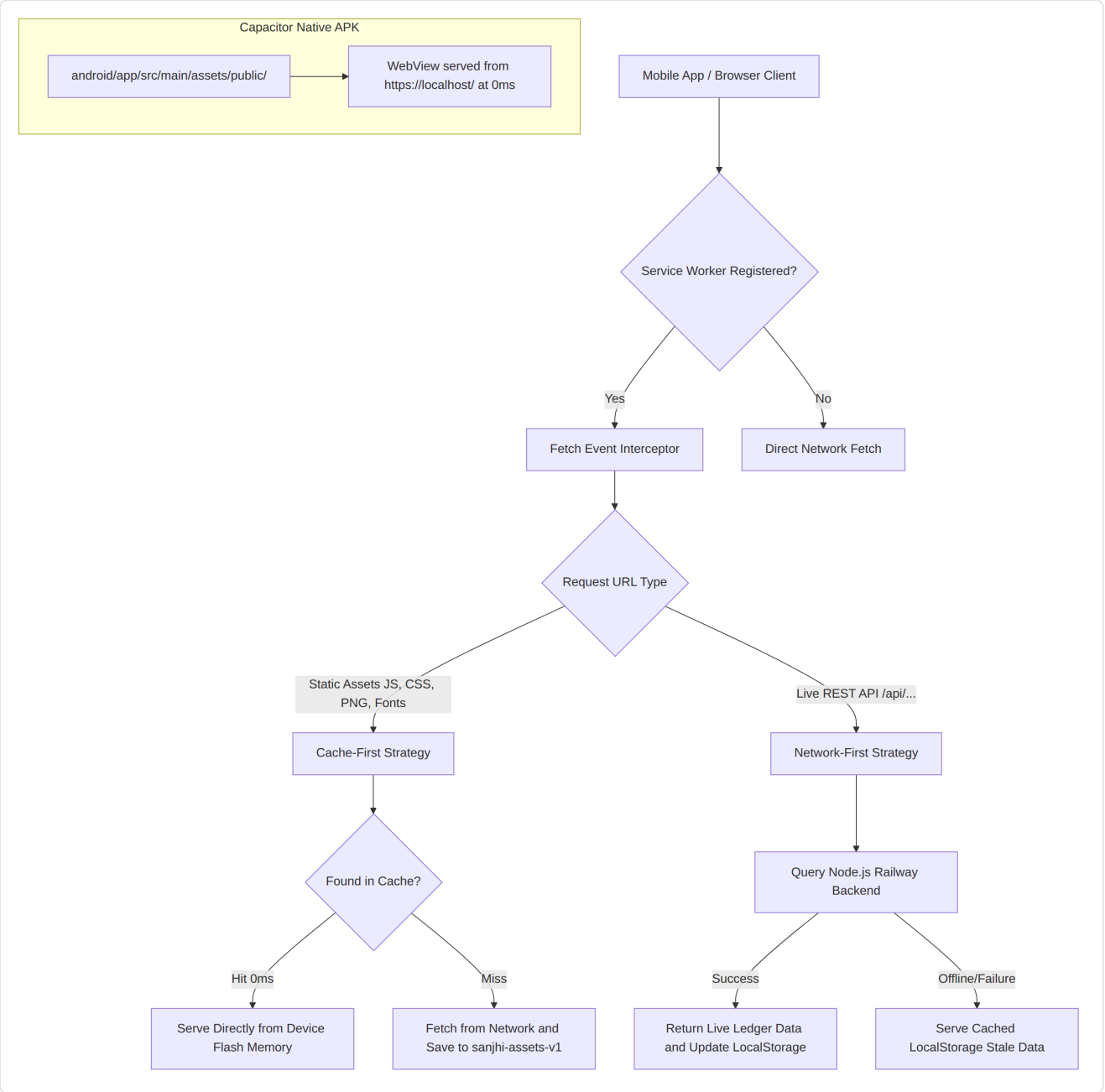

Figure 14.1 — Service Worker fetch interception: cache-first for static assets, network-first with stale-cache fallback for live API data, and the Capacitor native APK's locally bundled WebView path.

15. Future Roadmap — Consolidated View

Combining the original SRS Section 7 roadmap with the production gap analysis in Section 9.4 above, the following mind map consolidates every deferred or future-scope item across both documents into one traceable roadmap.

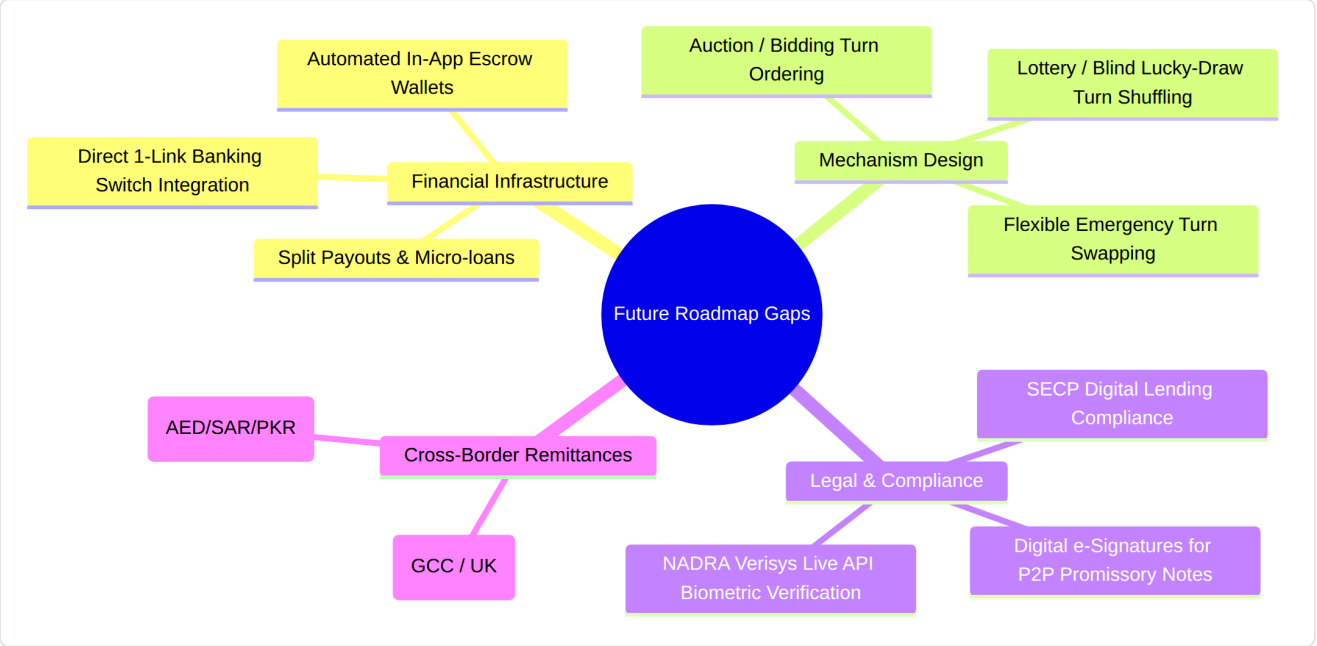

Figure 15.1 — Consolidated future roadmap across Financial Infrastructure, Mechanism Design, Legal & Compliance, and Cross-Border Remittances.

16. Summary of Key Takeaways

1. **Production-Ready Beyond Specification** — Sanjhi's current codebase does not just implement the original SRS; it modernizes the entire concept of ROSCAs with autonomous AI agents, WhatsApp voice intelligence, and native mobile wallet interoperability.
2. **Deterministic Ledger Security** — Financial trust is established through strict double-entry ledger bookkeeping, tamper-evident audit logs, and automatic trust-score adjustment, consistent with the confidentiality and role-scoping rules laid out in the original FR-LEDGER-01 and Section 4.2.
3. **Resilient Offline Performance** — With dual PWA caching and Capacitor native binary bundling, the app loads with near-zero delay regardless of cellular coverage, extending the original NFRs without contradicting them.
4. **Traceable Evolution, Not Divergence** — Every production extension documented in Sections 9–15 builds on an existing SRS requirement (FR-TRUST, FR-SUPPORT, FR-WABOT, Section 4.3) rather than replacing it — the original SRS remains the contractual baseline; this update layer records how far implementation has gone beyond it.